Lecture 16 – distance vector, bellman ford, BGP introduction

Bellman Ford Equation
$d_x(y)$ = cost of the least cost path from node x to node y

The Bellman-Ford equation

$$d_x(y) = \min_{v \in \text{neighbors}(x)} \left\{ c(x, v) + d_v(y) \right\}$$

→ the shortest path cost from x to y is the minimum over all neighbors c of the direct link cost to v plus v's shortest path cost to y

Dijkstra vs Bellman Ford

	Dijkstra (L15)	**Bellman-Ford (today)**
Information needed	**Full graph** at one place	Only **neighbor info**
Computation	Centralized	**Distributed**
Communication	Flood all links to everyone	Exchange DVs with neighbors
Negative weights	Not supported	Supported
Deployed as	OSPF, IS-IS	RIP

The key advantage of Bellman-Ford

No node needs the full topology. Each node only needs:

1. Its own direct link costs $c(x, v)$
2. Its neighbors' distance vectors $D_v(y)$ for all y

This enables a fully **distributed** algorithm.

Distance Vector Algorithm → each node x maintains a distance vector D_x=[D_x(y) for all y] → a vector of all the distances

Algorithm

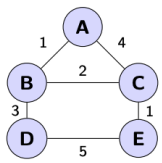
1. **Initialize**: $D_x(x) = 0$; $D_x(v) = c(x, v)$ for neighbors; $D_x(y) = \infty$ otherwise
2. **Send** D_x to all neighbors
3. **On receiving** D_v from neighbor v: for each destination y,
   if $c(x, v) + D_v(y) < D_x(y)$: update $D_x(y)$ and set next_hop$(y) = v$
4. If any $D_x(y)$ changed, **send updated** D_x to all neighbors
5. **Repeat** until no changes (convergence)

Properties:

- Iterative: updates continue until convergence
- Asynchronous: no global clock needed
- Distributed: each node talks only to direct neughbirs
- Self terminating: stops when no node has new information

Convergence Tree

Round 1



Each node knows only direct links:

	A	B	C	D	E
A	0	1	4	∞	∞
B	1	0	2	3	∞
C	4	2	0	∞	1
D	∞	3	∞	0	5
E	∞	∞	1	5	0

Rows = source node's distance vector.
Each node sends its DV to its neighbors.

Round 2

After receiving and processing neighbors' DVs:

	A	B	C	D	E	Changes
A	0	1	3	4	5	C: $4 \to 3$ via B; D: $\infty \to 4$ via B; E: $\infty \to 5$ via C
B	1	0	2	3	3	E: $\infty \to 3$ via C
C	3	2	0	5	1	A: $4 \to 3$ via B; D: $\infty \to 5$ via B
D	4	3	5	0	5	A: $\infty \to 4$ via B; C: $\infty \to 5$ via B
E	5	3	1	5	0	A: $\infty \to 5$ via C; B: $\infty \to 3$ via C

After second exchange — only one entry improves:

	A	B	C	D	E	Changes
A	0	1	3	4	4	E: $5 \to 4$ via B (heard $D_B(E) = 3$)
B	1	0	2	3	3	(no change)
C	3	2	0	5	1	(no change)
D	4	3	5	0	5	(no change)
E	4	3	1	5	0	A: $5 \to 4$ via C (heard $D_C(A) = 3$)

Round 3

Round 3: All nodes check — no improvements found. **Converged!**

Node A's forwarding table:

Destination	Next hop	Cost
B	B	1
C	B	3
D	B	4
E	B	4

All traffic from A goes via B (A's only low-cost neighbor).

Convergence → how many rounds ?

→ DV converges in at most |V| − 1 rounds

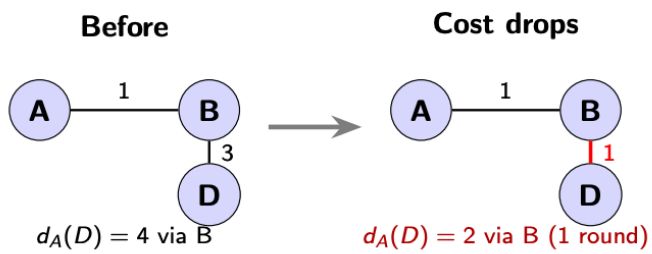
Compare with Dijkstra

Dijkstra needs the **full graph** but runs in one shot: $O(|V|^2)$ or $O(|E| \log |V|)$.
DV needs **multiple rounds** of neighbor exchange but each node only does local work.

For small graphs → usually converge in 2-3 rounds

Good News Travels Fast

When link cost decreases → improvem,ents propagate quickly

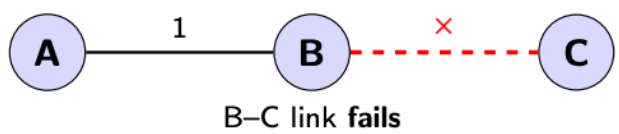


--> convergence in 1 round

The asymmetry

Good news travels fast. **Bad news travels slow.**

Bad news travels slow: count to infinity



1. B detects failure. Checks neighbor A: dA(C) = 2 and B computes: DB (C) = 1 + 2 = 3 via A. Wrong! A's route goes through B
2. B tells A: DB (C) = 3. A updates: DA(C) = 1 + 3 = 4
3. A tells B. B updates: DB (C) = 1 + 4 = 5. 4 Costs increment by 1 each round. . . counting to infinity

Root cause

B uses information from A **that depends on B itself**. A's route to C goes through B, but B doesn't know that. During convergence, packets for C **bounce between A and B** (routing loop).

Fixing count to infinity

<u>Solution 1:</u>

Rule: If A routes to C via B, then A does not advertise its route to C back to B

→ Don't tell your neighbor about a route that goes through them

<u>Solution 2:</u>

Rule: If A routes to C via B, then A tells B that dA(C) = ∞

→ Actively poison the reverse path → more robust than silence

Limitation

Split horizon and poisoned reverse fix **two-node loops**. They do **not** prevent count-to-infinity in loops with **three or more nodes**. Larger loops require additional mechanisms (hold-down timers, route poisoning).

LS vs DV: convergence summary

	Link-State (OSPF)	Distance-Vector (RIP)
Good news	Fast (one flood + recompute)	Fast (a few rounds)
Bad news	Fast (same mechanism)	**Slow (count-to-infinity)**
Loop-free?	Yes (full topology)	**Not guaranteed** during convergence

The defining weakness of distance-vector

Bad news travels slow. After a failure, DV may form routing loops that persist for many update rounds before converging. Link-state protocols avoid this because every router has the complete topology.

RIP: the simplest deployed DV protocol → Routing Information Protocol (RIP) implements the DV algorithm directly

Parameter	RIP choice
Metric	Hop count (every link costs 1)
Maximum distance	15 hops ($16 = \infty$)
Update interval	Every 30 seconds, full DV broadcast
Transport	UDP port 520
Triggered updates	Immediate update on route change
Timeout	180 seconds without update → set routes to ∞

Why one algorithm isnt enough

The internet has:

- Over 75,000 independently administered networks
- Hundreds of millions of IP prefixes
- Diverse policies, performance requirements, business relationships

Three barriers:

1. Scale: OSPF requires every router to store the full topology. With millions of routers, the LSDB and flooding traffic are unsustainable
2. Administrative autonomy: Different organizations need different policies. No single metric satisfies a university, a cloud provider, and a mobile carrier
3. Business relationships: Routing is driven by economics and contracts. An ISP may refuse to carry a competitor's traffic, even if the path is shorter.

Solution: hierarchical routing

Within an organization: OSPF or RIP (your choice).
Between organizations: a different protocol designed for policy — **BGP**.

Autonomous systems→ An autonomous system (AS) = a collection of IP networks and routers under one organization with a common routing policy

Type	Description	Example	Transit?
Stub	Single upstream connection	Small company	No
Multi-homed	Multiple ISPs, redundancy	University (e.g., BU)	No
Transit	Carries traffic between ASes	Comcast, AT&T	Yes

Each AS has a unique AS number (ASN): originally 16-bit (0–65,535), now 32-bit

	Intra-AS (IGP)	**Inter-AS (BGP)**
Scope	Within one AS	Between ASes
Goal	Shortest/cheapest path	Enforce routing **policy**
Protocols	OSPF, RIP, IS-IS	BGP
Metric	Link cost, hop count	Business relationships
Convergence	Seconds	**Minutes**

What is BGP → The Border Gateway Protocol (BGP) is the inter-AS routing protocol of the internet. Every AS that connects to the internet runs BGP
3 jobs:

- Advertise reachability: "My AS can reach these IP prefixes"
- Propagate paths: "To reach 128.197.0.0/16, go through AS-path [7922, 3356, 111]"
- Enforce policy: "I will carry this traffic" or "I refuse"

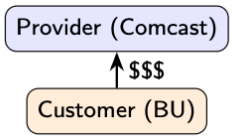
BGP is often called "the glue that holds the internet together"

Without BGP, the 75,000+ autonomous systems would be isolated islands. Every website you visit, every video you stream, relies on BGP routing your packets across multiple AS boundaries.

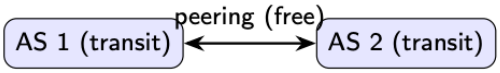
BGP and business relationships

Unlike OSPF or RIP, BGP routing is driven by business relationships:

Customer to Provider: Customer pays provider for internet access. Provider advertises customer routes globally



Peer to peer: Two ASes carry each other's traffic for free — but only for their own customers, not transit



The valley-free rule

Traffic flows **up** (customer → provider), across a **peering link** (at most once), then **down** (provider → customer). A well-configured BGP system never routes customer→provider→peer→provider→customer.

Why BGP is hard

1. Policy over optimality: the "best" route is not the shortest — it's the one that satisfies business contracts
2. No global view: unlike OSPF, no single entity sees the full routing table with all policies
3. Slow convergence: BGP convergence after a failure can take minutes (vs. seconds for OSPF)
4. Scale: the global BGP table contains over 1 million prefixes

5. Trust based: ASes trust their neighbors' announcements — limited verification infrastructure

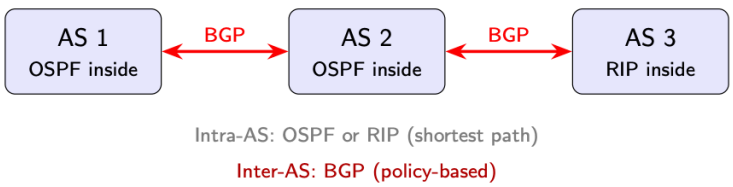
BGP incidents

Pakistan/YouTube (2008): Pakistan Telecom announced a more-specific prefix for YouTube. The announcement propagated globally, black-holing YouTube for ∼2 hours.

Facebook (2021): A config change withdrew BGP routes for all Facebook properties. DNS servers were behind those routes → domain names unresolvable. Outage: ∼6 hours.

The routing landscape

Lecture	Algorithm	Type	Scope	Deployed as
L15	Dijkstra	Link-state	Intra-AS	OSPF, IS-IS
L16	Bellman-Ford	Distance-vector	Intra-AS	RIP
L16	Path vector	Policy-based	Inter-AS	BGP



Intra-AS: OSPF or RIP (shortest path)

Inter-AS: BGP (policy-based)

LS vs DV: when to use each

Consider...	**Link-State (OSPF)**	**Distance-Vector (RIP)**
Network size	Large (100s of routers)	Small (<15 hops)
Convergence	Fast	Slow (count-to-infinity)
Memory/CPU	Higher (stores full topology)	Lower (stores only DV)
Complexity	More complex to implement	Simpler
Metric flexibility	Rich (bandwidth, delay)	Typically hop count only